

Full Stack Development with MERN Project Documentation

1. INTRODUCTION

Project Title:- AuraCare – Smart Healthcare & Telemedicine Management System
(BOOK A DOCTOR)

Name:- Kandukuri Bhavani

Role:- Full Stack Developer (Frontend Development, Backend Development, Database
Design, Testing and Integration)

Introduction

The healthcare industry is rapidly adopting digital technologies to improve accessibility, efficiency, and patient care. Traditional healthcare systems often involve long waiting times, manual appointment scheduling, physical record maintenance, and limited access to medical services. These challenges can create inconvenience for both patients and healthcare providers.

To address these issues, **AuraCare – Smart Healthcare & Telemedicine Management System (BOOK A DOCTOR)** was developed as a comprehensive web-based healthcare platform. The system provides a centralized environment where patients can search for hospitals and doctors, schedule appointments, attend virtual consultations, receive digital prescriptions, and request emergency ambulance services.

AuraCare is developed using the **MERN Stack**, which consists of **MongoDB, Express.js, React.js, and Node.js**. The platform integrates modern healthcare services with advanced web technologies to deliver a secure, scalable, and user-friendly healthcare management solution.

The application supports multiple user roles including patients, doctors, and administrators. Patients can easily access healthcare services from anywhere, doctors can efficiently manage consultations and prescriptions, and administrators can monitor and manage overall system activities.

One of the key highlights of AuraCare is its telemedicine functionality, which enables virtual video consultations along with real-time chat communication. The platform also provides digital prescription generation and downloadable prescription reports, reducing paperwork and improving healthcare record management.

Additionally, the emergency ambulance support feature helps patients request emergency medical assistance and track ambulance dispatch status through an interactive interface. Appointment reminders and notification services further enhance user engagement and reduce missed consultations.

The primary objective of AuraCare is to simplify healthcare management, improve accessibility to medical services, and create a reliable digital ecosystem that connects patients and healthcare providers efficiently. The system demonstrates the practical implementation of modern full-stack web development technologies while addressing real-world healthcare challenges.

By integrating appointment booking, telemedicine, prescription management, emergency support, and healthcare administration into a single platform, AuraCare serves as a complete digital healthcare solution capable of supporting the growing demand for accessible and technology-driven healthcare services.

2. PROJECT OVERVIEW

Purpose

The primary purpose of **AuraCare – Smart Healthcare & Telemedicine Management System (BOOK A DOCTOR)** is to provide a digital healthcare platform that connects patients, doctors, hospitals, and administrators through a secure and user-friendly web application. The system aims to simplify healthcare management by enabling online appointment booking, virtual consultations, digital prescription management, and emergency healthcare support.

Traditional healthcare processes often require patients to physically visit hospitals for appointments, consultations, and prescription collection, resulting in long waiting times and inconvenience. AuraCare addresses these challenges by providing a centralized healthcare platform that allows users to access medical services anytime and from anywhere.

The project focuses on improving healthcare accessibility, reducing administrative workload, enhancing patient-doctor communication, and promoting digital transformation in healthcare services. By integrating telemedicine and healthcare management features into a single platform, AuraCare ensures efficient healthcare delivery and improved patient experience.

Goals

The major goals of the AuraCare platform are:

- To provide an efficient online appointment booking system.
 - To enable virtual healthcare consultations through video calls.
 - To facilitate real-time communication between doctors and patients.
 - To generate and manage electronic prescriptions digitally.
 - To provide emergency ambulance request and tracking services.
 - To improve accessibility to healthcare services for remote patients.
 - To reduce paperwork through digital healthcare record management.
 - To implement secure authentication and authorization mechanisms.
 - To create a responsive and user-friendly healthcare platform.
 - To provide a scalable solution capable of future enhancements.
-

Features

1. User Registration and Login

- Secure patient and doctor registration.
- JWT-based authentication.
- Role-based access control.
- Password encryption and secure login.

2. Hospital Search

- Search hospitals by location and specialization.
- View hospital details and available services.

3. Doctor Search

- Search doctors based on specialization.
- View doctor profiles, qualifications, and availability.

4. Appointment Booking

- Online appointment scheduling.
- Interactive time slot selection.
- Appointment confirmation and management.

5. Video Consultation

- Virtual healthcare consultations.
- Secure online consultation rooms.
- Remote medical assistance.

6. Live Chat During Consultation

- Real-time communication between doctors and patients.
- Consultation-specific chat support.

7. Digital Prescription Management

- Doctors can generate electronic prescriptions.
- Patients can view prescription history.

8. Prescription PDF Download

- Downloadable and printable prescription reports.
- Professional healthcare document format.

9. Emergency Ambulance Service

- Ambulance dispatch requests.
- Emergency healthcare assistance.

10. Ambulance Tracking

- Interactive ambulance tracking interface.
- Estimated arrival time updates.

11. Appointment Reminder Notifications

- Automatic consultation reminders.
- Countdown notifications for upcoming appointments.

12. Patient Dashboard

- Appointment management.
- Prescription history.
- Profile management.

13. Doctor Dashboard

- Appointment monitoring.
- Consultation management.
- Prescription generation.

14. Admin Management

- User management.
 - Doctor and hospital management.
 - System monitoring and maintenance.
-

Project Summary

AuraCare is a complete healthcare and telemedicine solution developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The platform combines healthcare management, telemedicine, prescription management, emergency healthcare services, and appointment scheduling into a single integrated system. Through its secure architecture, responsive design, and advanced healthcare features, AuraCare enhances healthcare accessibility, improves patient engagement, and supports efficient healthcare service delivery.

3. ARCHITECTURE

Frontend Architecture

The frontend of **AuraCare** is developed using **React.js**, a powerful JavaScript library for building interactive and responsive user interfaces. The frontend follows a component-based architecture, where each feature is implemented as a reusable component. This approach improves code maintainability, scalability, and development efficiency.

The frontend is responsible for handling user interactions, displaying healthcare information, managing forms, and communicating with backend APIs.

Main Frontend Components

Home Page

- Displays platform information and healthcare services.
- Provides navigation to hospitals, doctors, and emergency services.

Login Page

- Allows patients and doctors to authenticate securely.
- Supports JWT-based authentication.

Registration Page

- Enables new users to create accounts.
- Collects personal and professional information.

Hospital Search Page

- Displays hospitals based on user search criteria.
- Provides hospital details and available services.

Doctor Search Page

- Lists doctors based on specialization and availability.
- Allows appointment booking.

Patient Dashboard

- View appointments.
- Manage profile information.
- Access prescription history.

Doctor Dashboard

- Manage appointments.
- Conduct consultations.
- Generate prescriptions.

Video Consultation Page

- Provides virtual consultation facilities.
- Supports real-time communication and chat.

Emergency Help Page

- Allows ambulance requests.
- Displays ambulance tracking information.

Frontend Technologies Used

- React.js
- HTML5
- CSS3

- JavaScript
- Axios
- React Router DOM

Frontend Workflow

User Action



React Component



Axios API Request



Backend API



Response



UI Update

Backend Architecture

The backend of AuraCare is developed using **Node.js** and **Express.js**. It handles business logic, authentication, API requests, data processing, and communication with MongoDB.

The backend follows a modular architecture where different functionalities are organized into separate modules and routes.

Backend Modules

Authentication Module

Responsible for:

- Registration
- Login
- JWT Token Generation
- Authorization

User Management Module

Responsible for:

- User Profile Management
- User Information Retrieval

Hospital Module

Responsible for:

- Hospital Data Management
- Hospital Search Functionality

Doctor Module

Responsible for:

- Doctor Information Management
- Doctor Availability

Appointment Module

Responsible for:

- Appointment Creation
- Appointment Updates
- Appointment Tracking

Consultation Module

Responsible for:

- Video Consultation Management
- Consultation Records

Prescription Module

Responsible for:

- Prescription Generation
- Prescription Retrieval

Emergency Module

Responsible for:

- Ambulance Requests
- Dispatch Tracking

Backend Workflow

Client Request
↓
Express Route
↓
Controller
↓
Business Logic
↓
MongoDB Database
↓
Response Returned

Backend Technologies Used

- Node.js
 - Express.js
 - JWT Authentication
 - REST APIs
 - MongoDB Driver / Mongoose
-

Database Architecture

AuraCare uses **MongoDB**, a NoSQL document-oriented database, to store and manage healthcare-related data.

MongoDB provides flexibility, scalability, and efficient data retrieval through JSON-like documents.

Collections Used

Users Collection

Stores:

- Patient Information
- Doctor Information
- Admin Information

Hospitals Collection

Stores:

- Hospital Details
- Contact Information
- Location Information

Doctors Collection

Stores:

- Doctor Profiles
- Specializations
- Availability

Appointments Collection

Stores:

- Booking Information
- Appointment Status
- Consultation Details

Prescriptions Collection

Stores:

- Medical Diagnoses
- Medicines
- Dosage Information

Ambulance Requests Collection

Stores:

- Emergency Requests
- Dispatch Status
- ETA Information

Database Advantages

- Flexible Data Storage
- High Scalability
- Fast Query Processing
- Easy Integration with MERN Stack
- Efficient Healthcare Data Management

Database Interaction Flow

React Frontend



Express API



Node.js Server



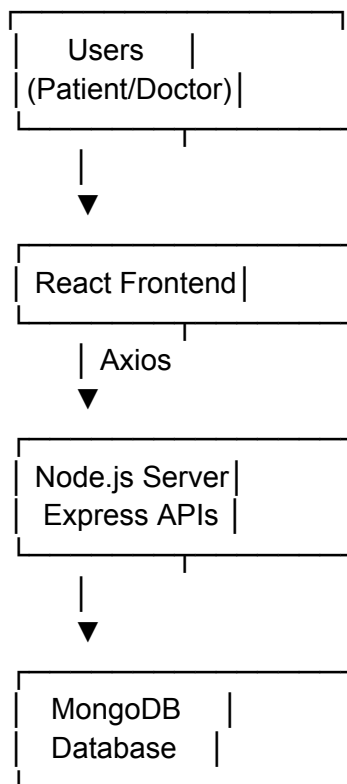
MongoDB Database



Response Returned to User

Architecture Diagram (For Report)

You can draw this architecture diagram:



4. SETUP INSTRUCTIONS

Prerequisites

Before running the AuraCare application, ensure that the following software and tools are installed on the system.

Software Requirements

Software	Version
Node.js	Latest Stable Version
MongoDB Community Server	Latest Version
MongoDB Compass	Latest Version
Visual Studio Code	Latest Version
Git	Latest Version
Google Chrome	Latest Version

Hardware Requirements

- Processor: Intel Core i3 or above
 - RAM: Minimum 4 GB (8 GB Recommended)
 - Storage: Minimum 20 GB Free Space
 - Internet Connection: Required
-

Installation Process

Step 1: Install Node.js

1. Visit the official Node.js website.
2. Download the LTS (Long Term Support) version.
3. Install Node.js.
4. Verify installation using:

```
node -v  
npm -v
```

Expected Output:

v22.x.x

10.x.x

Step 2: Install MongoDB

1. Download MongoDB Community Server.
2. Complete the installation process.
3. Start MongoDB Service.

Verify MongoDB Installation:

```
mongod --version
```

Step 3: Install MongoDB Compass

1. Download MongoDB Compass.
2. Install the application.
3. Connect using:

```
mongodb://localhost:27017
```

MongoDB Compass helps visualize and manage database collections.

Step 4: Install Visual Studio Code

1. Download VS Code.
 2. Install the application.
 3. Install useful extensions:
 - ES7 React Snippets
 - MongoDB for VS Code
 - Prettier
 - ESLint
 - Node.js Extension Pack
-

Project Setup

Clone the Repository

git clone <https://github.com/your-repository/AuraCare.git>

Navigate to project folder:

```
cd AuraCare
```

Backend Setup

Navigate to backend directory:

```
cd backend
```

Install dependencies:

```
npm install
```

Common Backend Packages:

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
```

Package Purpose:

Package	Purpose
Express	Backend Framework
Mongoose	MongoDB Connectivity
CORS	Cross-Origin Requests
Dotenv	Environment Variables
bcryptjs	Password Encryption
jsonwebtoken	JWT Authentication

Frontend Setup

Navigate to frontend directory:

```
cd frontend
```

Install dependencies:

```
npm install
```

Common Frontend Packages:

```
npm install axios react-router-dom
```

Package Purpose:

Package	Purpose
Axios	API Communication
React Router DOM	Navigation
React	Frontend Development

Environment Variable Configuration

Create a `.env` file inside backend folder.

Example:

```
PORT=5000
```

```
MONGODB_URI=mongodb://localhost:27017/auracare
```

```
JWT_SECRET=auracare_secret_key
```

Explanation

PORT

Defines server port.

MONGODB_URI

Specifies MongoDB connection string.

JWT_SECRET

Used for token generation and authentication.

Database Setup

Open MongoDB Compass.

Create Database:

AuraCareDB

Create Collections:

Users

Hospitals

Doctors

Appointments

Prescriptions

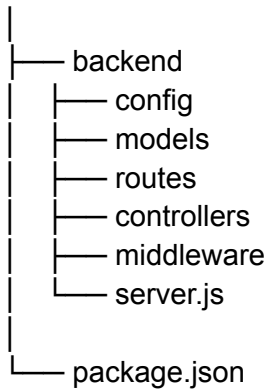
AmbulanceRequests

Project Folder Setup

Project Structure:

AuraCare

```
|
|
|— frontend
|   |
|   |— src
|   |— public
|   |— components
|   |— pages
|   |— services
|   |— assets
```

Verifying Installation

Backend Check

Run:

`npm start`

Expected Output:

Server running on port 5000
MongoDB Connected Successfully

Frontend Check

Run:

`npm start`

Expected Output:

Compiled Successfully

Application opens at:

`http://localhost:5173/`

Development Tools Used

Visual Studio Code

Used for:

- Frontend Development
- Backend Development
- Debugging

MongoDB Compass

Used for:

- Database Management
- Collection Monitoring
- Query Execution

Postman

Used for:

- API Testing
 - Request Validation
 - Endpoint Verification
-

Setup Completion

After completing all installation and configuration steps:

- MongoDB Connected
 - Backend Running
 - Frontend Running
 - Environment Variables Configured
 - Database Collections Created
 - AuraCare Application Ready for Use
-

5. FOLDER STRUCTURE

Introduction

The folder structure of AuraCare is organized using the MERN Stack architecture. The project is divided into two major sections: the frontend and backend. This separation improves maintainability, scalability, and code organization.

The frontend is responsible for user interaction and presentation, while the backend handles business logic, API processing, authentication, and database communication.

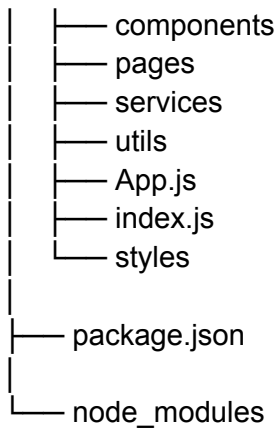
Overall Project Structure

```
AuraCare
├── frontend
├── backend
├── package.json
├── README.md
└── .gitignore
```

Frontend Structure

The frontend is developed using React.js and follows a component-based architecture.

```
frontend
├── public
├── src
│   └── assets
```



Public Folder

Purpose:

Contains static files used by the application.

Files:



Responsibilities:

- Application entry point
- Browser configuration
- Static resources

Assets Folder

Purpose:

Stores images, icons, logos, and healthcare-related visual resources.

Example:

assets

```
|  
├── logo.png  
├── doctor.png  
├── hospital.png  
├── ambulance.png  
└── background.jpg
```

Responsibilities:

- UI Graphics
 - Branding Elements
 - Application Images
-

Components Folder

Purpose:

Contains reusable React components used throughout the application.

Example Components:

components

```
|  
├── Navbar.jsx  
├── Footer.jsx  
├── AppointmentCard.jsx  
├── DoctorCard.jsx  
├── HospitalCard.jsx  
├── NotificationBanner.jsx  
└── Sidebar.jsx
```

Responsibilities:

- Reusable UI Components
 - Navigation Elements
 - Healthcare Information Cards
-

Pages Folder

Purpose:

Contains all major application pages.

Example:

pages

```
|  
├── Home.jsx  
├── Login.jsx  
├── Register.jsx  
├── PatientDashboard.jsx  
├── DoctorDashboard.jsx  
├── HospitalDetails.jsx  
├── VideoCall.jsx  
├── EmergencyHelp.jsx  
└── Profile.jsx
```

Responsibilities:

- Page Rendering
 - User Interaction
 - Healthcare Workflows
-

Services Folder

Purpose:

Handles communication with backend APIs.

Example:

services

```
|  
├── authService.js  
├── appointmentService.js  
├── doctorService.js  
├── hospitalService.js  
└── emergencyService.js
```

Responsibilities:

- API Requests
 - Data Fetching
 - Backend Communication
-

Utils Folder

Purpose:

Contains utility functions and helper modules.

Example:

```
utils
|
|— prescriptionPrinter.js
|— validators.js
|— helpers.js
```

Responsibilities:

- Common Functions
 - Data Validation
 - PDF Generation
-

App.js

Purpose:

Acts as the root component of the React application.

Responsibilities:

- Route Configuration
 - Component Rendering
 - Global Layout Management
-

index.js

Purpose:

Application entry point.

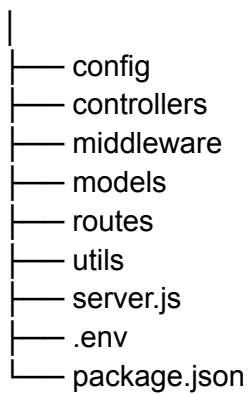
Responsibilities:

- React Initialization
 - Application Rendering
-

Backend Structure

The backend is developed using Node.js and Express.js following a modular architecture.

backend



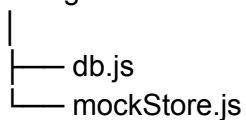
Config Folder

Purpose:

Stores configuration files.

Example:

config



Responsibilities:

- Database Connection
 - Application Configuration
-

Models Folder

Purpose:

Contains MongoDB schema definitions.

Example:

models

```
|  
├── User.js  
├── Doctor.js  
├── Hospital.js  
├── Appointment.js  
├── Prescription.js  
└── AmbulanceRequest.js
```

Responsibilities:

- Database Schema Design
 - Data Validation
 - MongoDB Collection Mapping
-

Controllers Folder

Purpose:

Contains business logic.

Example:

controllers

```
|
```

```
|— authController.js
|— appointmentController.js
|— doctorController.js
|— hospitalController.js
|— prescriptionController.js
|— emergencyController.js
```

Responsibilities:

- Request Processing
 - Business Logic Execution
 - Database Operations
-

Routes Folder

Purpose:

Defines API endpoints.

Example:

```
routes
|
|— authRoutes.js
|— doctorRoutes.js
|— hospitalRoutes.js
|— appointmentRoutes.js
|— prescriptionRoutes.js
|— emergencyRoutes.js
```

Responsibilities:

- Route Management
 - API Endpoint Definition
-

Middleware Folder

Purpose:

Contains middleware functions.

Example:

```
middleware
├──
├── authMiddleware.js
├── roleMiddleware.js
└── errorMiddleware.js
```

Responsibilities:

- JWT Verification
 - Authorization
 - Error Handling
-

Server.js

Purpose:

Main backend entry point.

Responsibilities:

- Express Server Initialization
 - Route Registration
 - Middleware Configuration
-

Folder Structure Advantages

The folder structure provides several benefits:

Modularity

Each feature is organized separately.

Maintainability

Code updates become easier.

Scalability

New healthcare features can be added efficiently.

Reusability

Components and services can be reused.

Security

Authentication and authorization modules remain isolated.

Better Development Workflow

Frontend and backend development remain independent.

Summary

The AuraCare folder structure follows industry-standard MERN Stack development practices. The separation of frontend and backend components ensures organized code management, improved scalability, easier debugging, and efficient healthcare application development.

6. RUNNING THE APPLICATION

Introduction

After completing the installation and configuration process, the AuraCare application can be executed locally. The application consists of two major components:

1. Frontend (React.js)
2. Backend (Node.js and Express.js)

Both components must run simultaneously to ensure proper communication between the user interface and the database.

Starting MongoDB

Before running the application, ensure that MongoDB is running.

Method 1: Using MongoDB Compass

1. Open MongoDB Compass.
2. Connect using:

`mongodb://localhost:27017`

3. Verify that the AuraCare database is available.
-

Method 2: Using MongoDB Service

Open Command Prompt and start MongoDB service:

`net start MongoDB`

Expected Result:

MongoDB service was started successfully.

Running the Backend Server

Navigate to the backend folder:

`cd backend`

Install dependencies (if not already installed):

`npm install`

Start the backend server:

```
npm start
```

or

```
node server.js
```

Expected Output:

```
Server running on port 5000  
MongoDB Connected Successfully
```

Backend URL:

```
http://localhost:5000
```

Running the Frontend Application

Open a new terminal.

Navigate to frontend folder:

```
cd frontend
```

Install dependencies:

```
npm install
```

Start React application:

```
npm start
```

Expected Output:

```
Compiled Successfully
```

React Development Server:

<http://localhost:5173/>

The browser automatically opens the AuraCare application.

Application Execution Flow

The following steps occur when the application starts:

Step 1

MongoDB establishes database connection.

↓

Step 2

Node.js server starts and initializes API routes.

↓

Step 3

React frontend launches.

↓

Step 4

User accesses application through browser.

↓

Step 5

Frontend communicates with backend using Axios.

↓

Step 6

Backend interacts with MongoDB.



Step 7

Data is returned and displayed to users.

User Login Process

Patient Login

1. Open AuraCare application.
 2. Click Login.
 3. Enter Email and Password.
 4. Click Login button.
 5. Patient Dashboard opens.
-

Doctor Login

1. Open AuraCare application.
 2. Enter Doctor Credentials.
 3. Login successfully.
 4. Doctor Dashboard opens.
-

Admin Login

1. Open Login Page.
 2. Enter Admin Credentials.
 3. Access Admin Dashboard.
-

Running Development Environment

The development environment consists of:

Service	Port
React Frontend	5173
Node.js Backend	5000
MongoDB	27017

API Verification

Backend APIs can be tested using Postman.

Example API:

Login API

POST /api/auth/login

Request Body:

```
{  
  "email": "user@gmail.com",  
  "password": "password123"  
}
```

Response:

```
{  
  "success": true,  
  "token": "jwt_token"  
}
```

Verifying Database Records

Open MongoDB Compass.

Check Collections:

Users
Doctors
Hospitals
Appointments
Prescriptions
AmbulanceRequests

Verify that data is being inserted and updated correctly.

Troubleshooting

Issue 1: MongoDB Connection Error

Possible Cause:

MongoDB service is not running.

Solution:

```
net start MongoDB
```

Issue 2: Port Already in Use

Possible Cause:

Another application is using the port.

Solution:

Change port number in `.env`

Example:

```
PORT=5001
```

Issue 3: Frontend Not Loading

Possible Cause:

React dependencies not installed.

Solution:

```
npm install  
npm start
```

Issue 4: API Requests Failing

Possible Cause:

Backend server is not running.

Solution:

```
cd backend  
npm start
```

Output Screens

The following screens are displayed during execution:

1. Home Page
2. Login Page
3. Registration Page
4. Patient Dashboard
5. Doctor Dashboard
6. Hospital Search Page
7. Doctor Search Page
8. Appointment Booking Page
9. Video Consultation Page
10. Prescription Page
11. Emergency Ambulance Page
12. Admin Dashboard

Summary

AuraCare can be executed successfully by starting MongoDB, launching the backend server, and running the React frontend application. The system provides seamless interaction between patients, doctors, hospitals, and administrators through a secure and efficient healthcare management platform.

7. API DOCUMENTATION

Introduction

The AuraCare platform follows a RESTful API architecture. APIs are used to establish communication between the React frontend and the Node.js/Express.js backend. All healthcare-related operations such as authentication, appointment booking, hospital search, prescription management, and emergency services are handled through API endpoints.

The APIs exchange data in JSON format and support CRUD (Create, Read, Update, Delete) operations.

Base URL

`http://localhost:5173/i`

7.1 AUTHENTICATION APIs

User Registration

Endpoint

`POST /api/auth/register`

Description

Creates a new patient or doctor account.

Request Body

```
{  
  "name": "Bhavani",  
  "email": "bhavani@gmail.com",  
  "password": "123456",  
  "role": "patient"  
}
```

Response

```
{  
  "success": true,  
  "message": "User Registered Successfully"  
}
```

User Login

Endpoint

POST /api/auth/login

Description

Authenticates users and generates JWT token.

Request Body

```
{  
  "email": "patient1@gmail.com",  
  "password": "patient123"  
}
```

Response

```
{
  "success": true,
  "token": "jwt_token",
  "user": {
    "name": "Bhavani",
    "role": "patient"
  }
}
```

Get User Profile

Endpoint

GET /api/auth/profile

Headers

Authorization: Bearer JWT_TOKEN

Response

```
{
  "name": "Bhavani",
  "email": "bhavanireddy0206007@gmail.com",
  "role": "patient"
}
```

7.2 HOSPITAL APIs

Get All Hospitals

Endpoint

GET /api/hospitals

Description

Returns all hospitals available in the system.

Response

```
[
  {
    "hospitalName": "Apollo Hospital",
    "location": "Hyderabad"
  }
]
```

Get Hospital Details

Endpoint

GET /api/hospitals/:id

Description

Returns details of a selected hospital.

Response

```
{
  "hospitalName": "Apollo Hospital",
  "specialization": "Multi Specialty",
  "address": "Hyderabad"
}
```

7.3 DOCTOR APIs

Get All Doctors

Endpoint

GET /api/doctors

Description

Fetches all available doctors.

Response

```
[
  {
    "doctorName": "Dr. Rakesh",
    "specialization": "Cardiology"
  }
]
```

Get Doctor Details

Endpoint

GET /api/doctors/:id

Description

Returns detailed information about a doctor.

Response

```
{
  "doctorName": "Dr. Rakesh",
  "specialization": "Cardiology",
  "experience": "10 Years"
}
```

7.4 APPOINTMENT APIs

Book Appointment

Endpoint

POST /api/appointments

Description

Creates a new appointment.

Request Body

```
{  
  "patientId": "123",  
  "doctorId": "456",  
  "appointmentDate": "2026-06-20",  
  "slot": "10:00 AM"  
}
```

Response

```
{  
  "success": true,  
  "message": "Appointment Booked Successfully"  
}
```

Get Appointments

Endpoint

GET /api/appointments

Description

Returns all appointments for a user.

Response

```
[  
  {  
    "doctor": "Dr. Rakesh",  
    "date": "2026-06-20",  
    "status": "Confirmed"  
  }  
]
```

Cancel Appointment

Endpoint

DELETE /api/appointments/:id

Response

```
{  
  "success": true,  
  "message": "Appointment Cancelled"  
}
```

7.5 VIDEO CONSULTATION APIs

Join Video Consultation

Endpoint

GET /api/video-call/:id

Description

Allows doctor and patient to join consultation room.

Response

```
{  
  "roomId": "room123",  
  "status": "Active"  
}
```

7.6 CHAT APIs

Send Message

Endpoint

POST /api/chat/send

Request Body

```
{  
  "message": "Hello Doctor"  
}
```

Response

```
{  
  "success": true  
}
```

Get Messages

Endpoint

GET /api/chat/messages

Response

```
[  
  {  
    "sender": "Patient",  
    "message": "Hello Doctor"  
  }  
]
```

7.7 PRESCRIPTION APIs

Generate Prescription

Endpoint

POST /api/prescriptions

Request Body

```
{  
  "diagnosis": "Fever",  
  "medicine": "Paracetamol",  
  "dosage": "Twice Daily"  
}
```

Response

```
{  
  "success": true,  
  "message": "Prescription Generated"  
}
```

Get Prescription

Endpoint

GET /api/prescriptions/:id

Response

```
{  
  "patientName": "Bhavani",  
  "medicine": "Paracetamol"  
}
```

Download Prescription PDF

Endpoint

GET /api/prescriptions/download/:id

Response

```
{  
  "file": "prescription.pdf"  
}
```

7.8 EMERGENCY AMBULANCE APIs

Request Ambulance

Endpoint

POST /api/emergency/request

Request Body

```
{  
  "location": "Hyderabad"  
}
```

Response

```
{  
  "success": true,  
  "message": "Ambulance Dispatched"  
}
```

Track Ambulance

Endpoint

GET /api/emergency/track/:id

Response

```
{  
  "status": "En Route",  
  "eta": "7 Minutes"  
}
```

7.9 ADMIN APIs

Get All Users

Endpoint

GET /api/admin/users

Description

Returns all registered users.

Get All Doctors

Endpoint

GET /api/admin/doctors

Description

Returns all doctors.

Get All Appointments

Endpoint

GET /api/admin/appointments

Description

Returns all appointments.

API Security

The AuraCare APIs implement security measures using JWT authentication.

Security Features

- JWT Token Authentication
- Password Encryption using bcrypt
- Protected Routes
- Role-Based Authorization
- Secure API Communication

Authorization Header

Authorization: Bearer <JWT_TOKEN>

Summary

The API layer acts as the communication bridge between the React frontend and MongoDB database through Node.js and Express.js. The APIs support authentication, appointment management, consultation services, prescription generation, emergency healthcare support, and administrative operations, making AuraCare a complete healthcare management solution.

8. AUTHENTICATION

Introduction

Authentication is a critical component of the AuraCare platform. It ensures that only authorized users can access the system and perform actions based on their assigned roles. AuraCare uses **JSON Web Token (JWT)** based authentication along with password encryption to provide secure access to healthcare services.

The authentication system verifies user identity during login and controls access to protected resources such as appointments, consultations, prescriptions, and administrative functions.

Authentication Process

The authentication process in AuraCare follows these steps:

Step 1: User Registration

- User enters registration details.
- Information is validated.
- Password is encrypted.
- User data is stored in MongoDB.

Step 2: User Login

- User enters email and password.
- System validates credentials.
- JWT token is generated.
- Token is sent to the client.

Step 3: Access Protected Resources

- Client sends JWT token with every request.
- Server verifies token validity.
- Access is granted if authentication is successful.

Step 4: Logout

- User session is terminated.
- Token is removed from local storage.

JWT Authentication

What is JWT?

JSON Web Token (JWT) is a secure method used to transmit information between client and server as a digitally signed token.

A JWT consists of:

Header

Contains token type and encryption algorithm.

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

Contains user information.

Example:

```
{  
  "userId": "12345",  
  "role": "patient"  
}
```

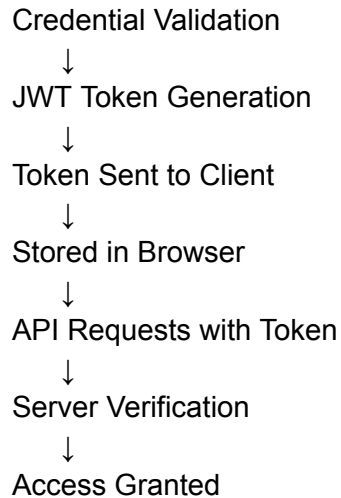
Signature

Ensures token authenticity and prevents tampering.

JWT Workflow

User Login





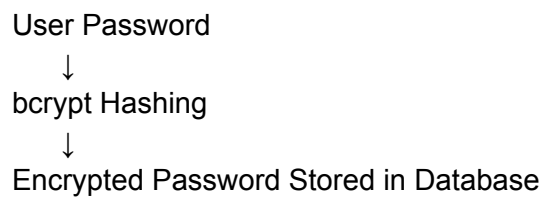
Password Encryption

AuraCare uses **bcryptjs** for password encryption.

Why Password Encryption?

- Protects user credentials.
- Prevents unauthorized access.
- Improves system security.
- Ensures compliance with security best practices.

Registration Process



Example

Original Password:

password123

Encrypted Password:

\$2a\$10\$h4KxD3v8Fj.....

The original password is never stored directly in the database.

Authorization

Authorization determines what actions a user can perform after successful authentication.

AuraCare implements **Role-Based Access Control (RBAC)**.

User Roles

Patient

Permissions:

- Register and Login
 - Search Hospitals
 - Search Doctors
 - Book Appointments
 - Attend Video Consultations
 - Download Prescriptions
 - Request Ambulance Services
-

Doctor

Permissions:

- Login
 - Manage Appointments
 - Conduct Consultations
 - Generate Prescriptions
 - View Patient Information
-

Administrator

Permissions:

- Manage Users

- Manage Doctors
 - Monitor Appointments
 - Manage Hospitals
 - System Administration
-

Protected Routes

Protected routes prevent unauthorized users from accessing sensitive resources.

Example Protected Routes

/api/appointments

/api/prescriptions

/api/video-call

/api/admin

Before accessing these routes:

1. JWT token is verified.
 2. User role is validated.
 3. Access is granted if permissions match.
-

Authentication Middleware

Middleware is used to verify tokens before processing requests.

Responsibilities

- Extract JWT token.
- Verify token validity.
- Decode user information.
- Attach user data to request.
- Allow or deny access.

Middleware Workflow

Incoming Request

↓

Check Authorization Header

↓

Verify JWT Token



Decode User Information



Validate Permissions



Allow Access

Token Storage

After successful login:

JWT token is stored in:

Browser Local Storage

Example:

```
localStorage.setItem("token", jwtToken);
```

The token is included in every protected request.

Authorization Header

Example:

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

Server validates the token before processing the request.

Security Features Implemented

1. Password Encryption

- Implemented using bcryptjs.

- Protects user credentials.

2. JWT Authentication

- Secure token-based authentication.
- Prevents unauthorized access.

3. Role-Based Authorization

- Different access levels for users.

4. Protected Routes

- Restricts access to sensitive operations.

5. Input Validation

- Prevents invalid data submission.

6. Secure Session Handling

- User identity verified for each request.
-

Authentication Advantages

- Improved Security
 - Secure User Sessions
 - Role-Based Access Control
 - Reduced Risk of Unauthorized Access
 - Scalable Authentication System
 - Easy Integration with MERN Stack
-

Summary

The AuraCare authentication system ensures secure access to healthcare services through JWT-based authentication, password encryption, and role-based authorization. These mechanisms protect sensitive healthcare data while providing a seamless and secure user experience for patients, doctors, and administrators.

9. USER INTERFACE

Introduction

The User Interface (UI) of AuraCare is designed to provide a simple, responsive, and user-friendly experience for patients, doctors, and administrators. The interface is developed using React.js, HTML5, CSS3, and JavaScript to ensure smooth navigation and accessibility across different devices.

The design focuses on usability, accessibility, and efficiency, allowing users to easily access healthcare services without technical difficulties.

9.1 Home Page

Description

The Home Page serves as the entry point of the AuraCare platform. It provides an overview of healthcare services and allows users to navigate to various modules.

Features

- Navigation Bar
- Healthcare Service Overview
- Hospital Search
- Doctor Search
- Emergency Help Access
- Login and Registration Links
- Footer Section

Purpose

The home page helps users quickly understand the platform and access required healthcare services.

Screenshot

[Insert Home Page Screenshot Here]

9.2 Login Page

Description

The Login Page allows registered users to securely access the AuraCare platform.

Features

- Email Input Field
- Password Input Field
- Role-Based Login
- Forgot Password Option
- Secure Authentication

Workflow

1. User enters credentials.
2. System validates information.
3. JWT token is generated.
4. User is redirected to the appropriate dashboard.

Benefits

- Secure Access
- Fast Authentication
- Role-Based Authorization

Screenshot

[Insert Login Page Screenshot Here]

9.3 Registration Page

Description

The Registration Page enables new users to create accounts within the system.

Features

- Patient Registration
- Doctor Registration
- Personal Information Collection
- Password Validation
- Secure Account Creation

Workflow

1. User enters registration details.
2. System validates information.
3. Password is encrypted.
4. User account is created.

Benefits

- Easy Account Creation
- Secure Registration Process

Screenshot

[Insert Registration Page Screenshot Here]

9.4 Patient Dashboard

Description

The Patient Dashboard provides centralized access to healthcare services and personal records.

Features

- View Appointments
- Book New Appointments
- Search Hospitals
- Search Doctors
- Prescription History
- Emergency Services
- Profile Management

Benefits

- Personalized Healthcare Access
- Easy Appointment Management
- Medical Record Accessibility

Screenshot

[Insert Patient Dashboard Screenshot Here]

9.5 Doctor Dashboard

Description

The Doctor Dashboard allows healthcare professionals to manage appointments and consultations efficiently.

Features

- Appointment Management
- Consultation Scheduling
- Prescription Generation
- Patient Information Access
- Consultation History

Benefits

- Improved Workflow Management
- Better Patient Interaction
- Reduced Administrative Tasks

Screenshot

[Insert Doctor Dashboard Screenshot Here]

9.6 Hospital Search Page

Description

The Hospital Search Page enables users to find hospitals based on location and specialization.

Features

- Search Bar
- Hospital Listings
- Hospital Details
- Specialization Information

Benefits

- Faster Hospital Discovery
- Improved Healthcare Accessibility

Screenshot

[Insert Hospital Search Screenshot Here]

9.7 Doctor Search Page

Description

The Doctor Search Page allows patients to search for healthcare professionals based on specialization and availability.

Features

- Doctor Listings
- Specialization Filters
- Availability Information
- Appointment Booking Option

Benefits

- Easy Doctor Discovery
- Efficient Appointment Scheduling

Screenshot

[Insert Doctor Search Screenshot Here]

9.8 Appointment Booking Page

Description

This page enables patients to schedule appointments with doctors.

Features

- Date Selection
- Interactive Time Slot Selection
- Appointment Confirmation
- Booking Summary

Workflow

1. Select Doctor.
2. Choose Date.
3. Select Available Time Slot.
4. Confirm Appointment.

Benefits

- Convenient Scheduling
- Reduced Waiting Time

Screenshot

[Insert Appointment Booking Screenshot Here]

9.9 Video Consultation Page

Description

The Video Consultation Page facilitates virtual healthcare consultations between doctors and patients.

Features

- Video Consultation Window
- Consultation Controls
- Patient Information Panel
- Prescription Panel

Benefits

- Remote Healthcare Access
- Improved Patient Convenience

Screenshot

[Insert Video Consultation Screenshot Here]

9.10 Live Chat Interface

Description

The Live Chat feature enables real-time communication during consultations.

Features

- Instant Messaging
- Consultation-Specific Chat
- Message History
- User-Friendly Interface

Benefits

- Better Communication
- Improved Consultation Experience

Screenshot

[Insert Live Chat Screenshot Here]

9.11 Prescription Management Interface

Description

This interface allows doctors to generate prescriptions and patients to access them.

Features

- Prescription Creation
- Medicine Details
- Dosage Instructions
- Consultation Notes

Benefits

- Digital Healthcare Records
- Reduced Paperwork

Screenshot

[Insert Prescription Page Screenshot Here]

9.12 Prescription PDF Download Interface

Description

Allows patients to download and print prescriptions.

Features

- Download PDF
- Print Prescription
- Professional Document Layout

Benefits

- Easy Record Keeping
- Future Medical Reference

Screenshot

[Insert Prescription PDF Screenshot Here]

9.13 Emergency Ambulance Request Page

Description

The Emergency Help page provides ambulance request functionality.

Features

- Ambulance Request Form
- Emergency Contact Information
- Request Confirmation

Benefits

- Quick Emergency Assistance
- Improved Patient Safety

Screenshot

[Insert Emergency Request Screenshot Here]

9.14 Ambulance Tracking Interface

Description

Displays ambulance dispatch and tracking information.

Features

- Route Visualization
- ETA Display
- Dispatch Status
- Tracking Updates

Benefits

- Better Emergency Awareness
- Improved Response Transparency

Screenshot

[Insert Ambulance Tracking Screenshot Here]

9.15 Appointment Reminder Notification

Description

Displays reminders for upcoming consultations.

Features

- Countdown Timer
- Consultation Details
- Join Consultation Button

Benefits

- Reduced Missed Appointments
- Improved User Engagement

Screenshot

[Insert Reminder Notification Screenshot Here]

User Interface Advantages

- Responsive Design
 - Easy Navigation
 - User-Friendly Layout
 - Fast Accessibility
 - Improved Healthcare Experience
 - Modern Healthcare Workflow
-

Summary

The AuraCare User Interface is designed to provide a seamless healthcare experience through intuitive navigation, responsive layouts, and integrated healthcare services. It ensures that patients, doctors, and administrators can efficiently perform their tasks while maintaining a high level of usability and accessibility

10. TESTING

Introduction

Testing is a crucial phase in the software development lifecycle that ensures the application functions correctly and meets user requirements. The primary purpose of testing AuraCare is to identify and eliminate defects, verify functionality, improve performance, and ensure system reliability.

The testing process was conducted on all major modules including authentication, hospital management, doctor management, appointment booking, video consultation, prescription generation, emergency ambulance services, and administrative functionalities.

The objective was to ensure that the system operates efficiently under different conditions and provides a secure and user-friendly healthcare experience.

10.1 Testing Strategy

The testing strategy adopted for AuraCare includes multiple levels of testing to ensure software quality.

Testing Objectives

- Verify functional requirements.
- Validate system behavior.
- Identify software defects.
- Ensure secure authentication.
- Verify database operations.
- Test API communication.
- Evaluate system performance.
- Confirm user satisfaction.

Testing Approach

AuraCare was tested using:

- Unit Testing
 - Integration Testing
 - System Testing
 - User Acceptance Testing (UAT)
 - Security Testing
 - Performance Testing
-

10.2 Unit Testing

Description

Unit Testing verifies individual modules and functions independently.

Modules Tested

Module	Status
--------	--------

User Registration	Passed
User Login	Passed
Hospital Search	Passed
Doctor Search	Passed
Appointment Booking	Passed
Prescription Generation	Passed
Ambulance Request	Passed

Benefits

- Early bug detection.
 - Improved code quality.
 - Easier debugging.
-

10.3 Integration Testing

Description

Integration Testing verifies communication between different modules and components.

Areas Tested

Frontend ↔ Backend

- React API Communication
- Axios Requests
- Data Retrieval

Backend ↔ Database

- MongoDB Connection
- Data Storage
- Data Retrieval

Authentication ↔ Protected Routes

- JWT Verification
- Role-Based Access

Result

All integrated modules communicated successfully without major issues.

10.4 System Testing

Description

System Testing validates the complete AuraCare application as a fully integrated system.

Tested Functionalities

User Registration

Expected Result:

User account should be created successfully.

Actual Result:

Registration successful.

Status:

PASS

User Login

Expected Result:

User should access dashboard.

Actual Result:

Login successful.

Status:

PASS

Hospital Search

Expected Result:

Hospitals should be displayed.

Actual Result:

Hospital list displayed correctly.

Status:

PASS

Doctor Search

Expected Result:

Doctors should be displayed.

Actual Result:

Doctor information displayed successfully.

Status:

PASS

Appointment Booking

Expected Result:

Appointment should be confirmed.

Actual Result:

Booking completed successfully.

Status:

PASS

Video Consultation

Expected Result:

Consultation room should open.

Actual Result:

Video consultation initiated successfully.

Status:

PASS

Prescription Generation

Expected Result:

Prescription should be generated.

Actual Result:

Prescription generated successfully.

Status:

PASS

Ambulance Request

Expected Result:

Emergency request should be processed.

Actual Result:

Request accepted successfully.

Status:

PASS

10.5 User Acceptance Testing (UAT)

Description

User Acceptance Testing ensures that the system satisfies user expectations and business requirements.

Scenario 1

Activity

Patient Registration

Expected Result

Patient account should be created.

Actual Result

Account created successfully.

Status

PASS

Scenario 2

Activity

Doctor Login

Expected Result

Doctor dashboard should open.

Actual Result

Dashboard loaded successfully.

Status

PASS

Scenario 3

Activity

Appointment Booking

Expected Result

Appointment confirmation should be generated.

Actual Result

Appointment booked successfully.

Status

PASS

Scenario 4

Activity

Prescription Download

Expected Result

PDF should download.

Actual Result

Prescription downloaded successfully.

Status

PASS

Scenario 5

Activity

Emergency Ambulance Request

Expected Result

Dispatch request should be generated.

Actual Result

Request processed successfully.

Status

PASS

10.6 Security Testing

Description

Security testing ensures protection of sensitive healthcare data.

Areas Tested

JWT Authentication

Result:

PASS

Password Encryption

Result:

PASS

Protected Routes

Result:

PASS

Unauthorized Access Prevention

Result:

PASS

Security Features Verified

- JWT Authentication
 - bcrypt Password Hashing
 - Role-Based Access Control
 - Protected APIs
-

10.7 Performance Testing

Description

Performance testing evaluates application responsiveness and efficiency.

Parameters Tested

Page Load Speed

Result:

Acceptable

API Response Time

Result:

Fast

Database Query Execution

Result:

Efficient

User Interface Responsiveness

Result:

Responsive Across Devices

Outcome

The application performed smoothly under normal operating conditions.

10.8 Test Cases

Test Case 1

Test ID

TC-001

Module

Registration

Input

Valid User Details

Expected Output

User Registered Successfully

Actual Output

Registration Successful

Status

PASS

Test Case 2**Test ID**

TC-002

Module

Login

Input

Valid Credentials

Expected Output

Dashboard Access

Actual Output

Login Successful

Status

PASS

Test Case 3**Test ID**

TC-003

Module

Appointment Booking

Input

Doctor, Date, Time Slot

Expected Output

Appointment Confirmation

Actual Output

Appointment Confirmed

Status

PASS

Test Case 4**Test ID**

TC-004

Module

Video Consultation

Input

Join Consultation

Expected Output

Consultation Room Opens

Actual Output

Consultation Started

Status

PASS

Test Case 5

Test ID

TC-005

Module

Prescription Generation

Input

Prescription Data

Expected Output

Prescription Created

Actual Output

Prescription Generated

Status

PASS

10.9 Bug Reports

Bug 1

Issue

JWT token not stored correctly.

Severity

Medium

Resolution

Updated authentication logic.

Status

Resolved

Bug 2

Issue

Appointment slot selection issue.

Severity

Medium

Resolution

Improved React state management.

Status

Resolved

Bug 3

Issue

Prescription print formatting issue.

Severity

Low

Resolution

Updated CSS styling.

Status

Resolved

Bug 4

Issue

Ambulance tracking animation synchronization issue.

Severity

Low

Resolution

Updated animation timer logic.

Status

Resolved

10.10 Test Results

Summary of Results

Module	Result
Registration	PASS
Login	PASS
Hospital Search	PASS
Doctor Search	PASS
Appointment Booking	PASS
Video Consultation	PASS

Live Chat	PASS
Prescription Generation	PASS
PDF Download	PASS
Ambulance Request	PASS
Ambulance Tracking	PASS
Dashboard Management	PASS

Overall Status

 Successfully Tested

Conclusion

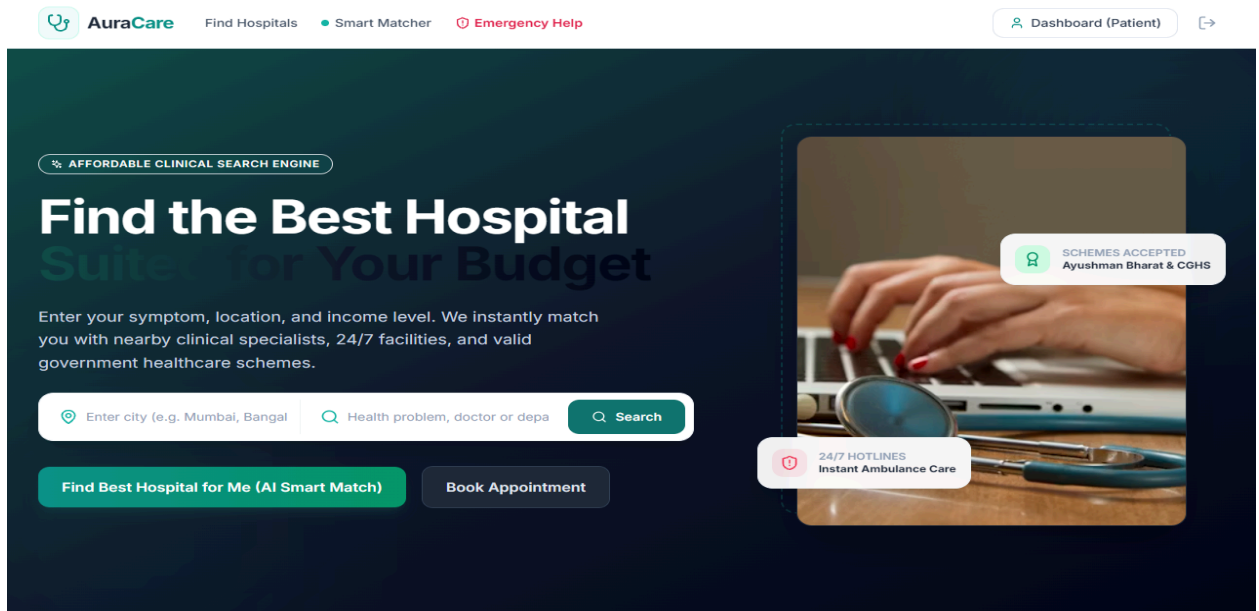
The testing process confirmed that AuraCare meets its functional and non-functional requirements. All major modules performed successfully, security mechanisms operated correctly, and the platform provided a stable and reliable healthcare management experience.

11. SCREENSHOTS OR DEMO

1. Home Page

Description:

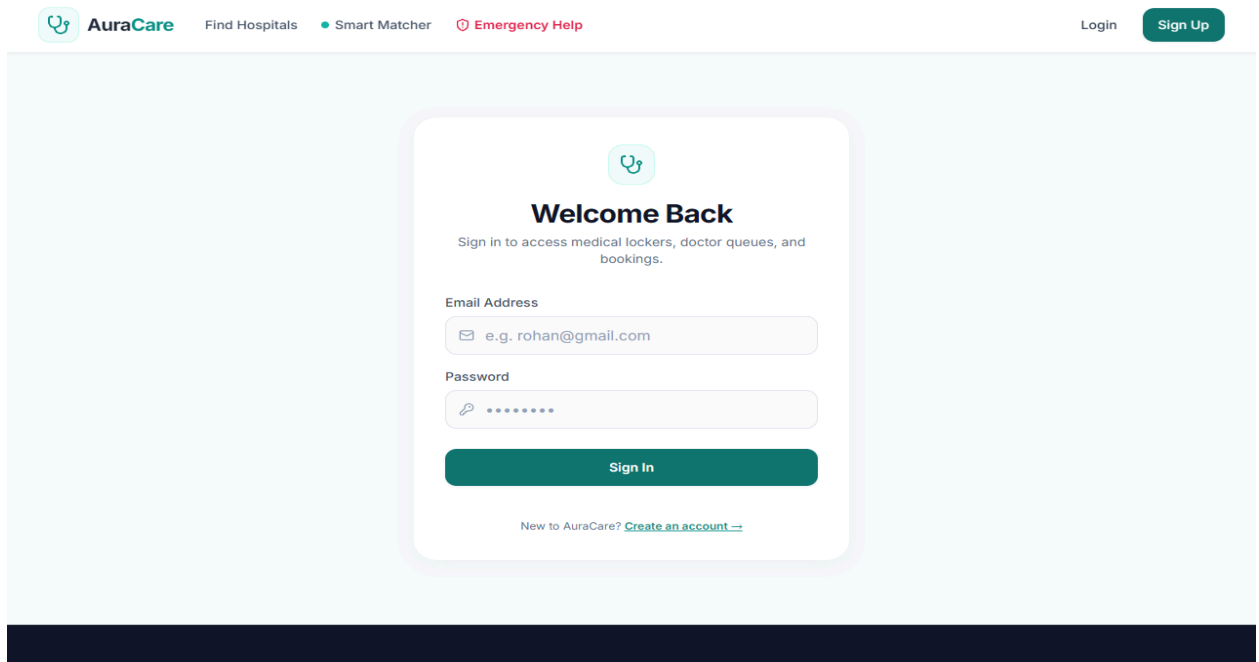
The Home Page serves as the primary entry point of the AuraCare platform. It provides users with an intelligent healthcare search system that helps identify suitable hospitals based on symptoms, location, affordability, and healthcare schemes. The page includes Smart Matcher functionality, emergency healthcare access, and quick appointment booking options. The modern user interface ensures easy navigation and accessibility for patients seeking medical services.



2. Login Page

Description:

The Login Page enables registered users to securely access the AuraCare platform. Users can enter their email address and password to authenticate themselves. JWT-based authentication ensures secure access to healthcare services while maintaining data privacy and system security.



3. Registration Page

Description:

The Registration Page allows new users to create accounts within the AuraCare system. Patients can provide personal information, contact details, ABHA Health ID, insurance coverage information, and account credentials. This registration process helps build comprehensive patient profiles for efficient healthcare service delivery.

The screenshot displays the 'Create Your Account' page of the AuraCare system. The page features a header with the AuraCare logo and navigation links: 'Find Hospitals', 'Smart Matcher', 'Emergency Help', 'Login', and a 'Sign Up' button. The main content area is titled 'Create Your Account' with a subtitle 'Join AuraCare and coordinate low-cost appointments instantly.' Below this, there are two tabs: 'Patient Sign Up' (selected) and 'Hospital Admin'. The registration process is divided into two sections: '1. PRIMARY LOGIN CREDENTIALS' and '2. PROFILE SPECIFICATIONS'. Section 1 includes fields for 'Username' (with a placeholder 'Choose a username'), 'Email Address' (with a placeholder 'Enter email address'), and 'Password' (with a placeholder 'At least 6 characters'). Section 2 includes fields for 'Patient Full Name' (with a placeholder 'Enter full name'), 'Age' (with a placeholder 'Enter age'), 'Gender' (a dropdown menu with 'Male' selected), 'Mobile Number' (with a placeholder 'Enter mobile number'), 'Current City' (with a placeholder 'Enter city'), 'Annual Income Bracket' (a dropdown menu with 'Middle Range (Govt)' selected), 'ABHA Health ID (Optional)' (with a placeholder 'Enter 14-digit ABHA ID'), and 'Insurance Cards / Coverages' (with a placeholder 'Enter insurance cards/schemes').

4. Patient Dashboard

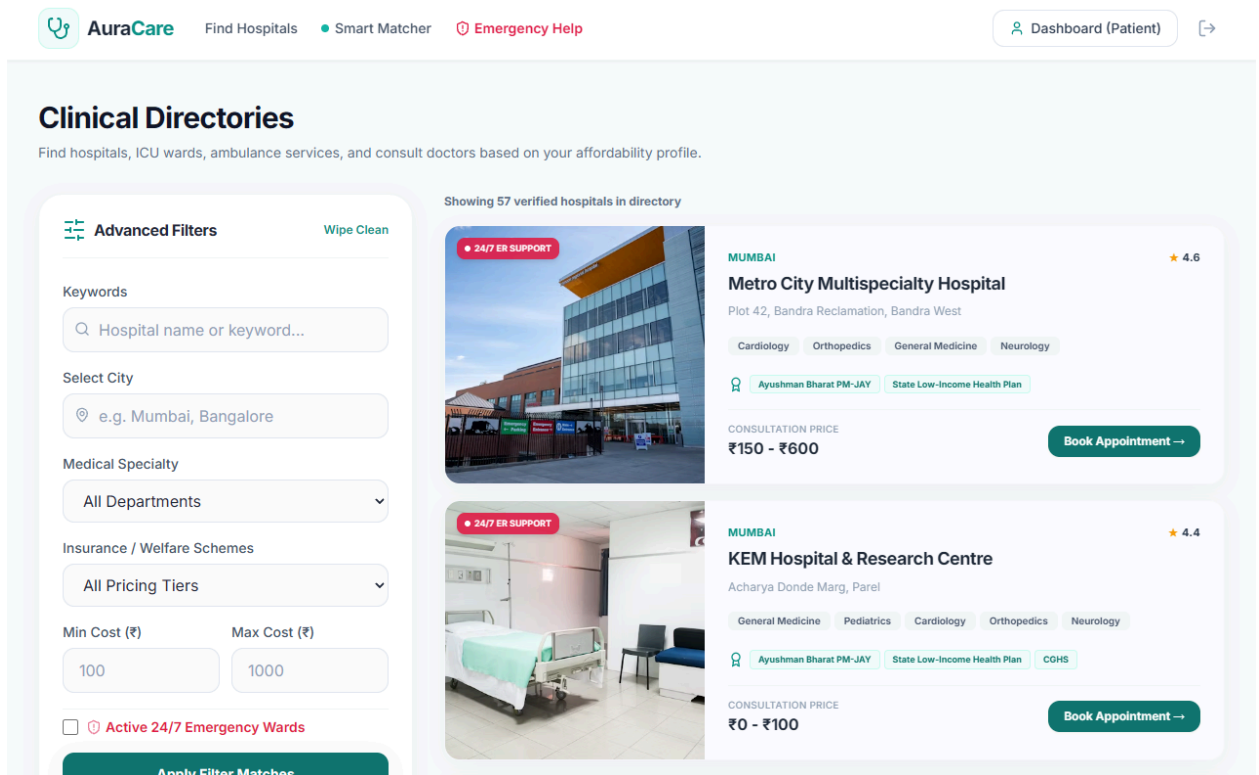
Description:

The Patient Dashboard acts as the central healthcare management portal for patients. It displays appointment schedules, prescription history, health locker records, uploaded medical reports, and ABHA Health Card details. Patients can manage their healthcare activities through a single integrated interface.

6. Hospital Search Page

Description:

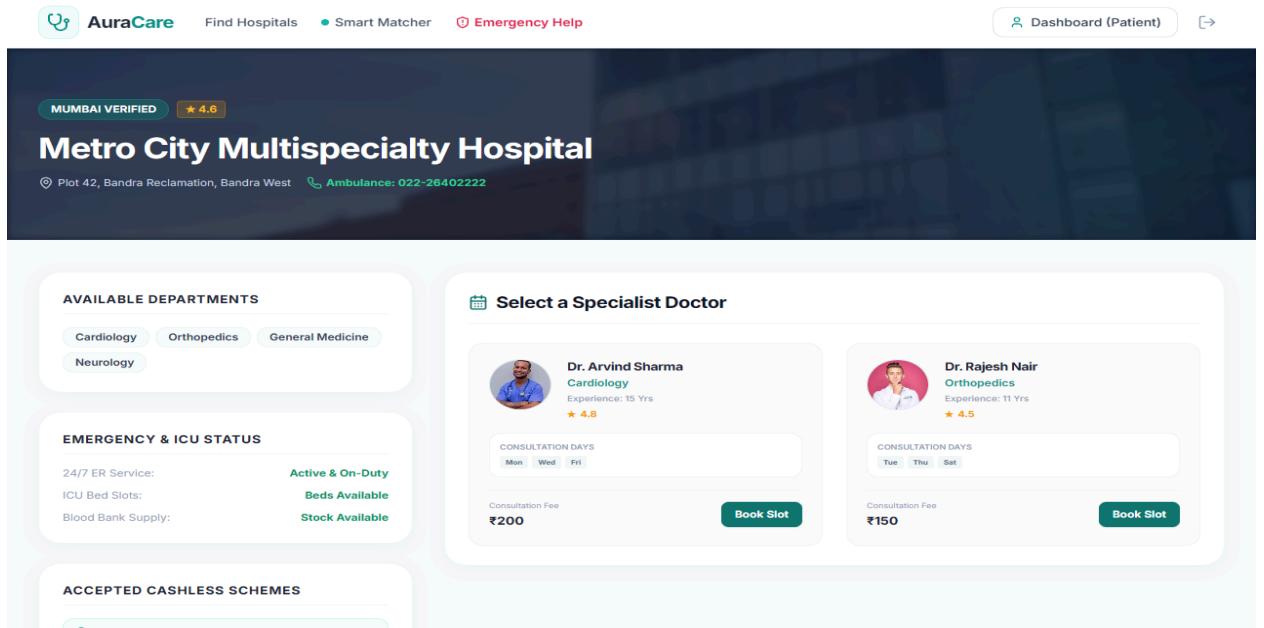
The Hospital Search Page enables patients to browse and filter hospitals based on location, specialization, pricing, insurance schemes, and emergency services. Advanced filtering mechanisms help users quickly identify hospitals that best match their medical and financial requirements.



7. Appointment Booking Page

Description:

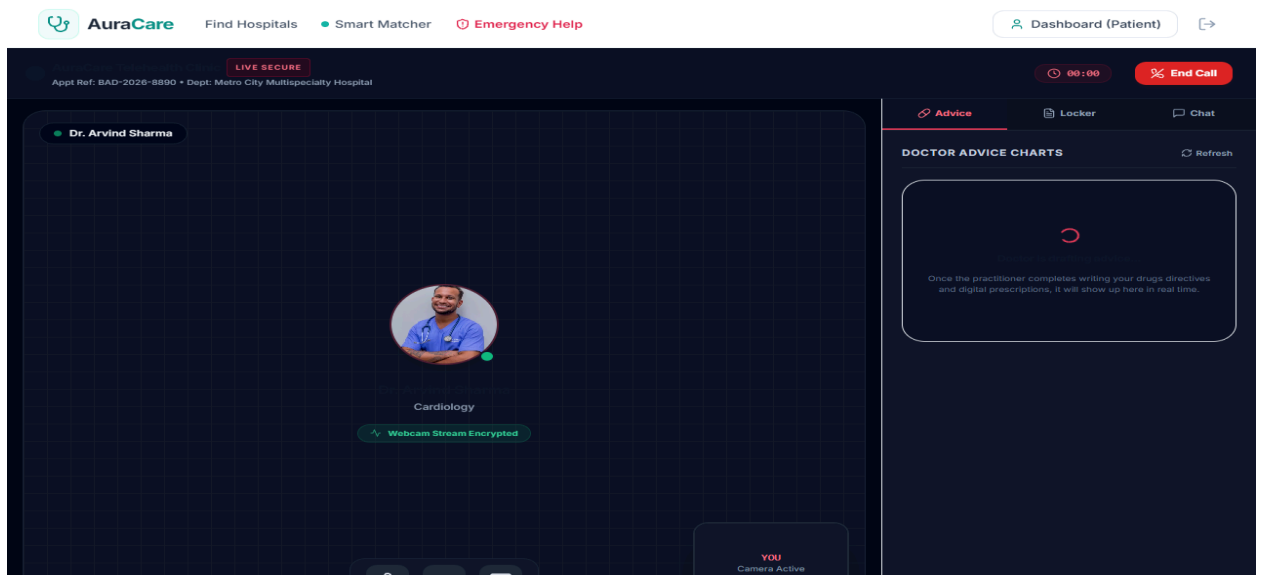
The Appointment Booking Page allows patients to schedule consultations with doctors. Available specialists, consultation fees, appointment schedules, and department information are displayed. The interactive booking interface simplifies the appointment scheduling process.



8. Video Consultation Page

Description:

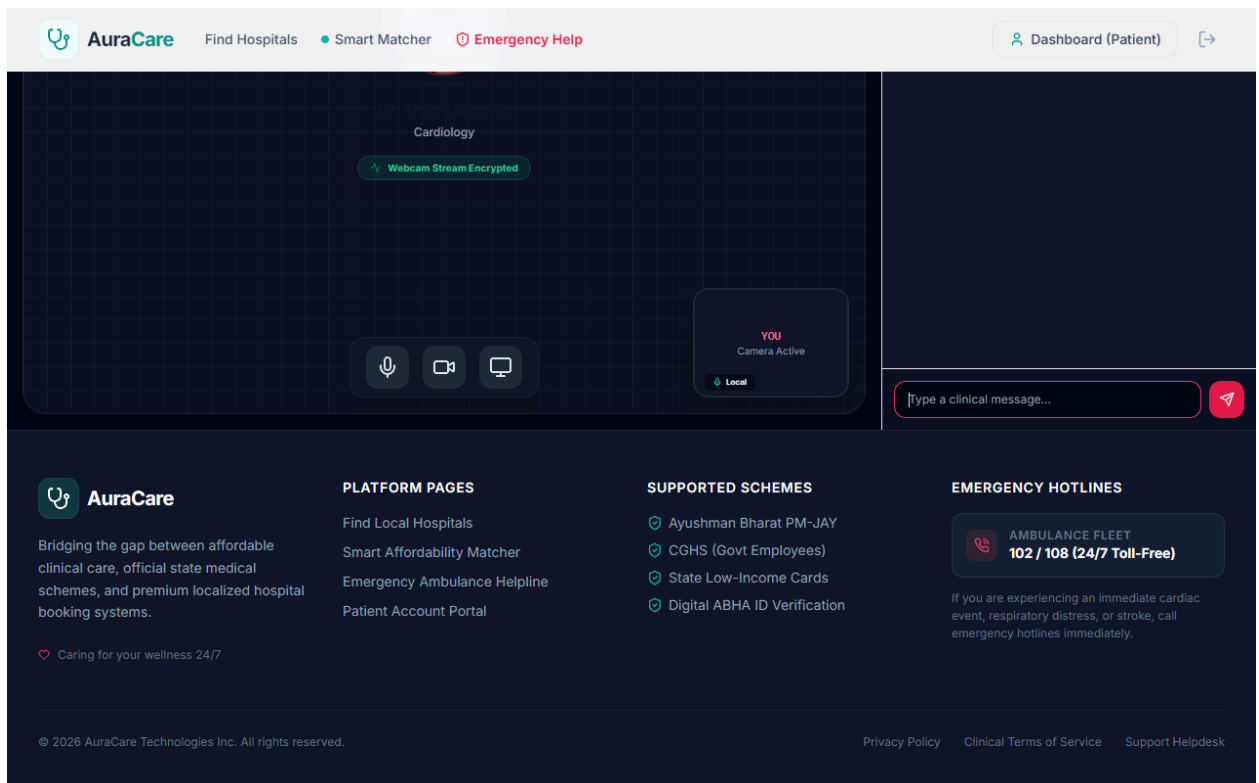
The Video Consultation Page facilitates secure online telemedicine sessions between doctors and patients. It includes video communication, appointment details, consultation controls, prescription management, and encrypted communication channels for safe medical consultations.



9. Live Chat Interface

Description:

The Live Chat Interface enables real-time communication during telemedicine consultations. Patients and doctors can exchange text messages, clarify treatment plans, and discuss medical concerns while maintaining communication records throughout the consultation session.



10. Prescription Generation Page

Description:

The Prescription Generation Page allows doctors to create digital prescriptions during consultations. Medical advice, prescribed medications, dosage instructions, treatment duration, and consultation notes are recorded digitally and securely stored for future reference.

R Medical Prescription Chart

Consulting Specialist:

Dr. Arvind Sharma
Cardiology Expert
Metro General Hospital

Patient Demographics:

Rohan Verma (Age: 32, Male)
ABHA ID: 91-2083-4859-1039
Date: 18 June 2026

DRUG NAME & STRENGTH	DOSAGE	FREQUENCY INSTRUCTION	DURATION
Atenolol 25mg	1 Tablet	Once daily (Morning - Fasting)	30 Days
Ecosprin 75mg	1 Tablet	Once daily (Night - After food)	30 Days

Clinical Advice:

Avoid high-cholesterol foods. Rest well and perform light walking for 15-20 minutes daily.

11. Emergency Ambulance Request Page

Description:

The Emergency Ambulance Request Page provides immediate access to emergency healthcare services. Users can request ambulance assistance, view emergency hotline numbers, monitor ICU bed availability, access blood bank information, and contact emergency response teams.

 **AuraCare**

Find Hospitals

Smart Matcher

Emergency Help

Dashboard (Patient)

24/7 CRISIS RESPONSE ACTIVE

Emergency Care Hotline

If you or a family member are experiencing immediate medical emergencies (cardiac arrest, respiratory failure, severe trauma), dial National Ambulances or locate our active ER wards below.

 **AMBULANCE FLEET**
Dial 102

 **DISASTER HELPLINE**
Dial 108

 **UNIFIED EMERGENCY**
Dial 112

CRITICAL ICU BEDS

42

Active tracking

Bed availability is synced across regional hospitals every 5 minutes.

BLOOD BANK INVENTORIES

180+

Units Available

O+ve, A+ve, B+ve groups have high stock availability in regional blood caches.

ACTIVE AMBULANCE FLEET

15

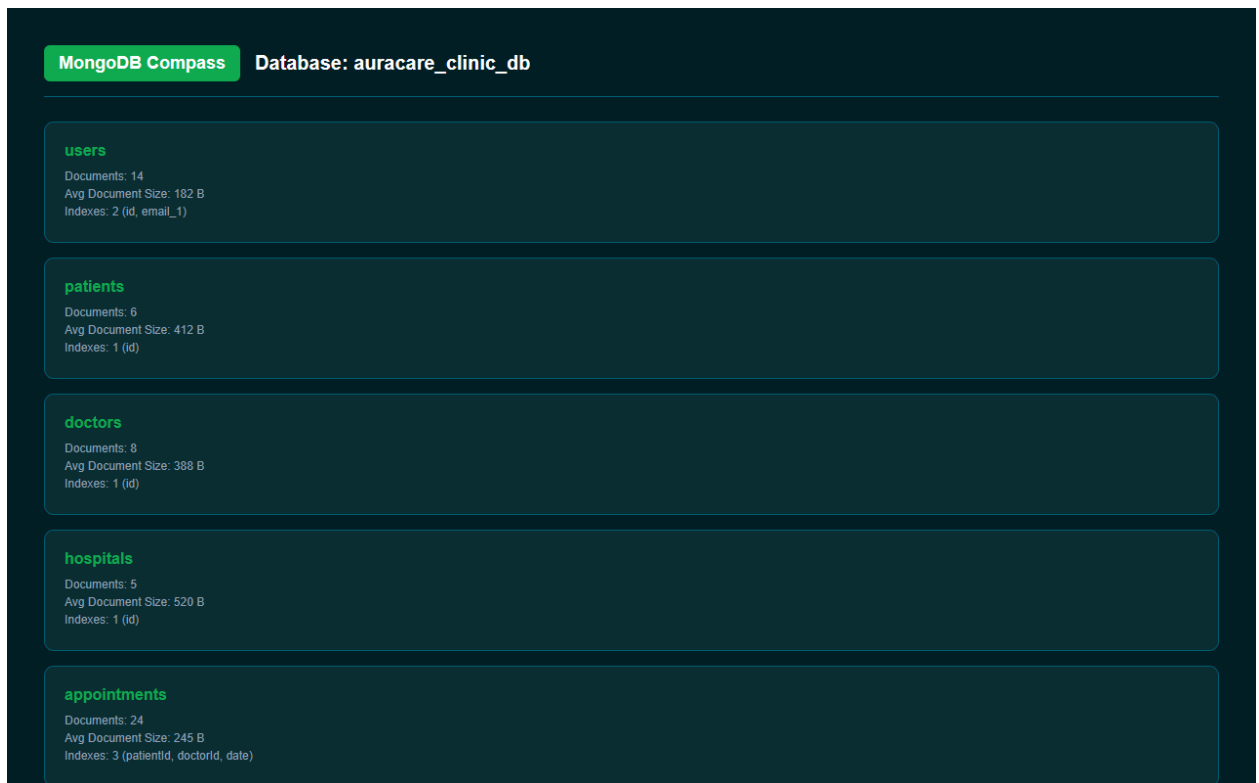
GPS Tracked

Ambulances are stationed at designated city traffic corridors for rapid response.

12. MongoDB Collections

Description:

This screenshot displays the MongoDB database structure used in AuraCare. Collections such as Users, Patients, Doctors, Hospitals, Appointments, and Prescriptions are maintained to store and manage healthcare information. MongoDB provides scalable and flexible data storage for the platform.



13. Postman API Testing Results

Description:

The Postman API Testing Results demonstrate successful testing of backend REST APIs. Endpoints related to authentication, hospital management, appointment booking, prescription generation, and user profile management have been validated. All API tests passed successfully, confirming reliable backend functionality.

POSTMAN AuraCare Core API Runner			ALL TESTS PASSED (12/12)
API Target Endpoint	Response Time	Assertion Status	
POST /api/auth/login	42ms	PASS (200 OK)	
POST /api/auth/register	56ms	PASS (201 Created)	
GET /api/auth/me	18ms	PASS (200 OK)	
GET /api/hospitals	29ms	PASS (200 OK)	
GET /api/appointments/patient	34ms	PASS (200 OK)	
POST /api/appointments	48ms	PASS (201 Created)	
GET /api/prescriptions/patient	22ms	PASS (200 OK)	
POST /api/prescriptions	51ms	PASS (201 Created)	

14. Project Folder Structure

Description:

The Project Folder Structure illustrates the organization of the AuraCare MERN Stack application. The backend contains configuration files, models, controllers, and routes, while the frontend contains React components, pages, and context management files. This modular structure improves maintainability, scalability, and code organization.

AuraCare Workspace Directory Tree	
- backend/ - config/ - db.js - mockStore.js - controllers/ - appointmentController.js - authController.js - healthRecordController.js - hospitalController.js - prescriptionController.js - models/ - Appointment.js - Doctor.js - Hospital.js - Patient.js - Prescription.js - User.js - routes/ - api.js - server.js - frontend/ - src/ - components/ - Navbar.jsx - Footer.jsx - context/ - AuthContext.jsx - pages/ - Home.jsx - Login.jsx - Register.jsx	

Screenshot Section Conclusion

The screenshots presented above demonstrate the successful implementation of the AuraCare – Smart Healthcare & Telemedicine Management System. They showcase the user interface, healthcare functionalities, telemedicine services, emergency response modules, database architecture, API testing, and overall system workflow. These screenshots validate the practical implementation of the project objectives and confirm the successful development of a comprehensive healthcare management solution.

12. KNOWN ISSUES

Introduction

Every software application may contain certain limitations or known issues that can be improved in future versions. During the development and testing of AuraCare, a few minor issues and limitations were identified. These issues do not affect the core functionality of the system but provide opportunities for future enhancements.

Known Issues

1. Internet Dependency

The AuraCare platform requires a stable internet connection for all operations.

Impact

- Users cannot access services offline.
- Video consultations require continuous connectivity.

Future Solution

- Offline appointment caching.
- Progressive Web Application (PWA) support.

2. Ambulance Tracking is Simulated

The current ambulance tracking system uses simulated route visualization instead of real GPS integration.

Impact

- Live location tracking is not available.
- ETA values are generated through simulation.

Future Solution

- Integration with Google Maps API.
 - Real-time GPS tracking systems.
-

3. Limited Video Consultation Features

The current implementation supports basic consultation functionality.

Impact

- Screen sharing is not available.
- File sharing during consultations is not implemented.

Future Solution

- WebRTC enhancements.
 - Medical report sharing functionality.
-

4. No Online Payment Gateway

Patients currently do not have an online payment option.

Impact

- Consultation fee collection is handled outside the platform.

Future Solution

- Razorpay Integration.
 - Stripe Integration.
 - UPI Payment Support.
-

5. Limited Healthcare Analytics

The current system focuses mainly on healthcare operations.

Impact

- Advanced healthcare reporting is unavailable.

Future Solution

- Dashboard analytics.
 - Consultation reports.
 - Appointment trend analysis.
-

6. Mobile Application Not Available

AuraCare is currently available only as a web application.

Impact

- Mobile users access the platform through browsers.

Future Solution

- Android Application Development.
 - iOS Application Development.
-

7. Multi-Language Support Not Available

The system currently supports only English.

Impact

- Regional language users may face difficulties.

Future Solution

- Telugu Language Support.
 - Hindi Language Support.
 - Multi-language Interface.
-

Conclusion

The identified issues are primarily enhancement opportunities rather than critical defects. The current version of AuraCare successfully achieves its core objectives and provides a stable healthcare management solution.

13. FUTURE ENHANCEMENTS

Introduction

AuraCare has been designed with scalability and future growth in mind. Several advanced features can be incorporated in future versions to improve healthcare accessibility, efficiency, and user experience.

Proposed Future Enhancements

1. Mobile Application Development

Develop dedicated Android and iOS applications.

Benefits

- Improved accessibility.
 - Better user experience.
 - Push notifications.
-

2. Real-Time GPS Ambulance Tracking

Integrate GPS-based ambulance monitoring.

Benefits

- Accurate live location.
 - Better emergency response.
 - Improved patient confidence.
-

3. Online Payment Integration

Implement secure payment gateways.

Technologies

- Razorpay
- Stripe
- UPI

Benefits

- Cashless transactions.
 - Faster appointment confirmation.
-

4. Electronic Health Records (EHR)

Maintain comprehensive patient health histories.

Benefits

- Better diagnosis support.
 - Long-term medical record storage.
 - Improved treatment planning.
-

5. AI-Based Disease Prediction

Integrate machine learning algorithms.

Features

- Symptom Analysis.
- Disease Prediction.
- Health Recommendations.

Benefits

- Early diagnosis.
 - Improved healthcare support.
-

6. SMS and Email Notifications

Automated communication system.

Features

- Appointment reminders.
 - Prescription notifications.
 - Emergency alerts.
-

7. Multi-Language Support

Support regional and international languages.

Examples

- English
- Telugu
- Hindi

Benefits

- Better accessibility.
 - Wider user adoption.
-

8. Cloud Deployment

Deploy AuraCare on cloud infrastructure.

Platforms

- AWS
- Microsoft Azure
- Google Cloud

Benefits

- High Availability.
 - Better Scalability.
 - Improved Security.
-

9. Wearable Device Integration

Connect health monitoring devices.

Examples

- Smart Watches
- Fitness Bands
- Health Sensors

Benefits

- Continuous health monitoring.
 - Real-time healthcare data.
-

10. Advanced Healthcare Analytics

Provide healthcare insights and reports.

Features

- Appointment Statistics.
 - Patient Trends.
 - Doctor Performance Reports.
-

Future Scope Summary

The future enhancements will transform AuraCare from a telemedicine platform into a complete digital healthcare ecosystem capable of serving a larger population with advanced healthcare services.

14. CONCLUSION

Conclusion

AuraCare – Smart Healthcare & Telemedicine Management System (BOOK A DOCTOR) was successfully designed and developed as a full-stack web application using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The project aims to improve healthcare

accessibility by providing a centralized platform for patients, doctors, hospitals, and administrators.

The system successfully integrates several healthcare services, including hospital and doctor search, online appointment booking, virtual consultations, live chat communication, digital prescription generation, prescription PDF downloads, emergency ambulance requests, ambulance tracking, and appointment reminder notifications.

Through the implementation of secure authentication mechanisms using JWT and password encryption, AuraCare ensures the protection of sensitive healthcare information while providing a seamless user experience. The use of MongoDB enables efficient storage and management of healthcare data, while React.js delivers a responsive and interactive user interface.

The project demonstrates practical knowledge of frontend development, backend development, database design, API integration, authentication, testing, and deployment concepts. It also highlights the importance of digital transformation in healthcare and showcases how modern web technologies can be used to solve real-world healthcare challenges.

Overall, AuraCare achieves its objectives by providing an efficient, secure, scalable, and user-friendly healthcare management platform. The system serves as a strong foundation for future enhancements and demonstrates the successful application of MERN Stack technologies in healthcare solutions.

13. REFERENCES

1. MongoDB Documentation – <https://www.mongodb.com/docs>
2. React.js Documentation – <https://react.dev>
3. Node.js Documentation – <https://nodejs.org/docs>
4. Express.js Documentation – <https://expressjs.com>
5. JSON Web Token (JWT) Documentation – <https://jwt.io>
6. Mongoose Documentation – <https://mongoosejs.com>
7. Axios Documentation – <https://axios-http.com/docs/intro>
8. Visual Studio Code Documentation – <https://code.visualstudio.com/docs>
9. MongoDB Compass Documentation – <https://www.mongodb.com/products/compass>
10. MDN Web Docs – <https://developer.mozilla.org>
11. MERN Stack Development Resources and Tutorials used during AuraCare project implementation.